



BABU BANARASI DAS UNIVERSITY
SCHOOL OF COMPUTER APPLICATIONS

NO SQL PROJECT

Student Management System Using MongoDB

Submitted To	Submitted By
Mr. Harendra Singh Rajpoot	Prince Kumar
Department BCA (DS & AI)	Roll No. 16
Class BCA DS 25	Academic Session 2026–2027
Date 25th September 2026	Project Student Management System Using MongoDB

MongoDB • NoSQL • CRUD • Query Operators • Sorting • Filtering

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher, Mr. Harendra Singh Rajpoot, for giving me the opportunity to work on this NoSQL project titled "Student Management System Using MongoDB".

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including roll number, name, age, marks and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting and searching records. Comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte have also been demonstrated.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and collection.
- To insert 50 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand AND and OR operations.

PROJECT OVERVIEW AND DATABASE STRUCTURE

The project models a simple student information system. The MongoDB database is named **Students** and the collection is named **students**. Each student document contains five main fields: roll, name, age, marks and city.

Field	Purpose	Example
roll	Unique student roll number	1
name	Student name	Anamika
age	Student age	19
marks	Student marks	88
city	Student city	Lucknow

TABLE OF CONTENTS

1. Introduction and Acknowledgement
2. Objectives
3. Project Overview and Database Structure
4. Complete Student Dataset
5. MongoDB Implementation and Operations
6. MongoDB Operators Used
7. Project Workflow
8. Conclusion
9. Project Details

Note: Following the supplied reference, the implementation section uses MongoDB execution screenshots rather than reproducing the commands as standalone code blocks. The screenshots show the command/output style used in the project.

COMPLETE STUDENT DATASET

The initial collection contains 50 student documents with the fields roll, name, age, marks and city. The following tables reproduce the complete dataset supplied in the reference project.

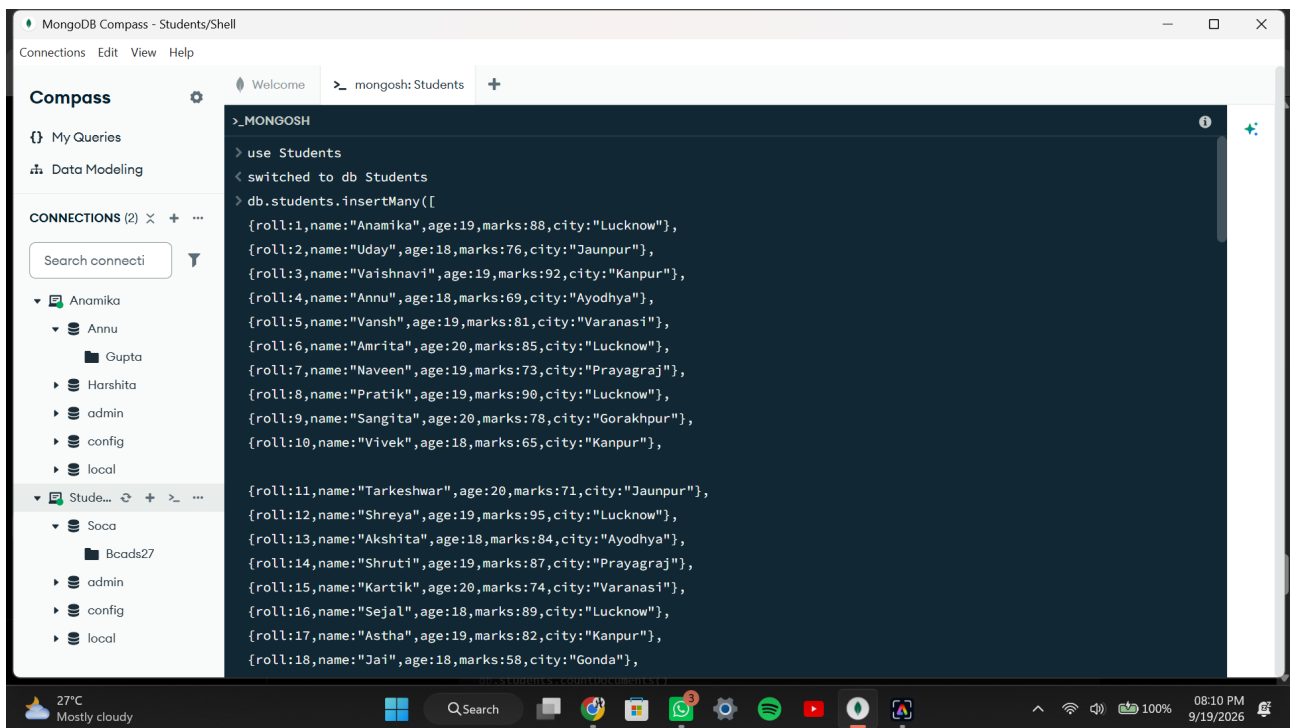
Roll	Name	Age	Marks	City
1	Anamika	19	88	Lucknow
2	Uday	18	76	Jaunpur
3	Vaishnavi	19	92	Kanpur
4	Annu	18	69	Ayodhya
5	Vansh	19	81	Varanasi
6	Amrita	20	85	Lucknow
7	Naveen	19	73	Prayagraj
8	Pratik	19	90	Lucknow
9	Sangita	20	78	Gorakhpur
10	Vivek	18	65	Kanpur
11	Tarkeshwar	20	71	Jaunpur
12	Shreya	19	95	Lucknow
13	Akshita	18	84	Ayodhya
14	Shruti	19	87	Prayagraj
15	Kartik	20	74	Varanasi
16	Sejal	18	89	Lucknow
17	Astha	19	82	Kanpur
18	Jai	18	58	Gonda
19	Harshita	20	91	Lucknow
20	Omkar	19	63	Bareilly
21	Ayush	18	77	Lucknow
22	Saumya	19	86	Jaunpur
23	Vidya	20	72	Kanpur
24	Adarsh	19	80	Ayodhya
25	Anmol	18	67	Prayagraj

COMPLETE STUDENT DATASET (CONTINUED)

Roll	Name	Age	Marks	City
26	Anushka	20	94	Lucknow
27	Ananya	19	88	Varanasi
28	Bhaskar	18	61	Gorakhpur
29	Rohan	19	75	Kanpur
30	Nishant	20	79	Lucknow
31	Vineet	18	68	Jaunpur
32	Anika	19	90	Lucknow
33	Jai	20	55	Ayodhya
34	Lakshya	18	83	Kanpur
35	Krishna	19	96	Prayagraj
36	Rumana	20	70	Lucknow
37	Priyanshu	18	81	Varanasi
38	Aditya	19	66	Gorakhpur
39	Dhruv	20	93	Lucknow
40	Nivedita	19	85	Kanpur
41	Meenu	18	74	Jaunpur
42	Akanksha	20	89	Lucknow
43	Sarika	19	62	Ayodhya
44	Anchal	18	78	Prayagraj
45	Umra	20	84	Lucknow
46	Nashra	19	73	Kanpur
47	Aman	18	59	Gonda
48	Priya	20	87	Lucknow
49	Rahul	19	64	Bareilly
50	Neha	18	91	Varanasi

1. CREATE DATABASE

The project starts by creating or switching to the Students database. This provides the database context in which the student collection is maintained.



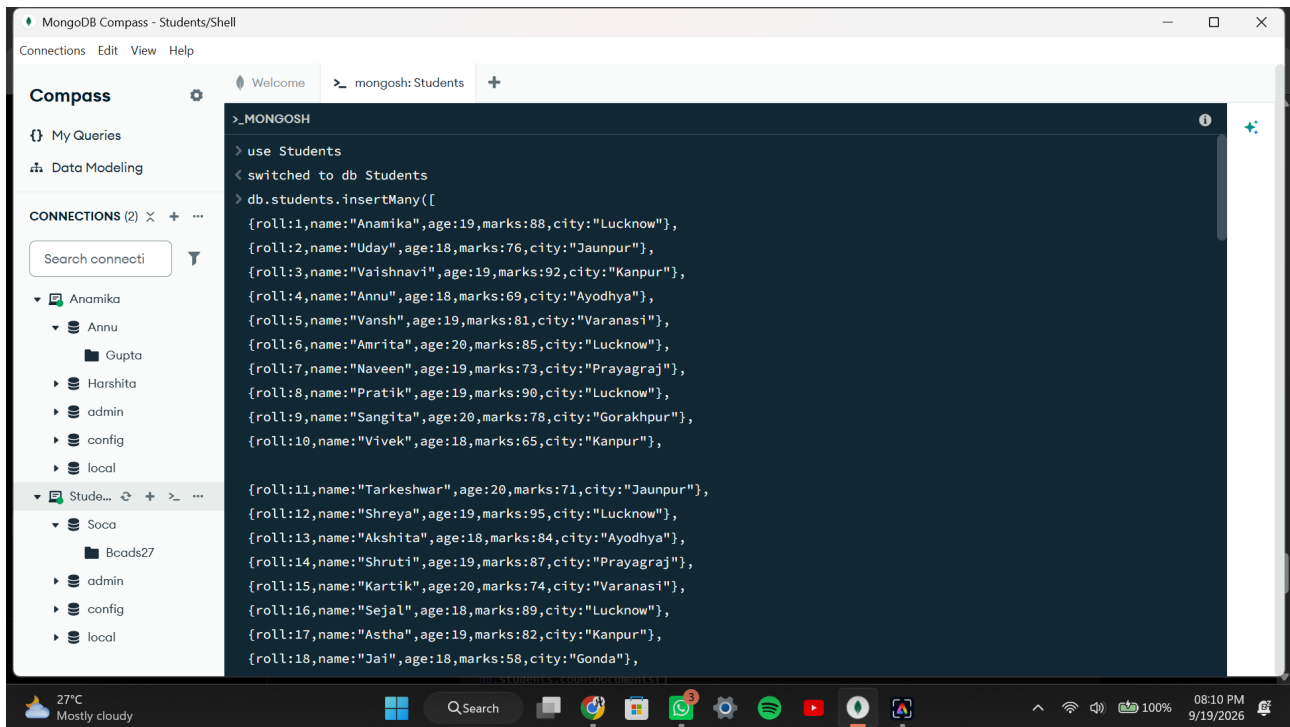
The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a tree view of databases. The 'Students' database is selected. The main panel shows the MongoDB Shell with the following commands and output:

```
> use Students
switched to db Students
> db.students.insertMany([
  {roll:1,name:"Anamika",age:19,marks:88,city:"Lucknow"},
  {roll:2,name:"Uday",age:18,marks:76,city:"Jaunpur"},
  {roll:3,name:"Vaishnavi",age:19,marks:92,city:"Kanpur"},
  {roll:4,name:"Annu",age:18,marks:69,city:"Ayodhya"},
  {roll:5,name:"Vansh",age:19,marks:81,city:"Varanasi"},
  {roll:6,name:"Amrita",age:20,marks:85,city:"Lucknow"},
  {roll:7,name:"Naveen",age:19,marks:73,city:"Prayagraj"},
  {roll:8,name:"Pratik",age:19,marks:90,city:"Lucknow"},
  {roll:9,name:"Sangita",age:20,marks:78,city:"Gorakhpur"},
  {roll:10,name:"Vivek",age:18,marks:65,city:"Kanpur"},
  {roll:11,name:"Tarkeshwar",age:20,marks:71,city:"Jaunpur"},
  {roll:12,name:"Shreya",age:19,marks:95,city:"Lucknow"},
  {roll:13,name:"Akshita",age:18,marks:84,city:"Ayodhya"},
  {roll:14,name:"Shruti",age:19,marks:87,city:"Prayagraj"},
  {roll:15,name:"Kartik",age:20,marks:74,city:"Varanasi"},
  {roll:16,name:"Sejal",age:18,marks:89,city:"Lucknow"},
  {roll:17,name:"Astha",age:19,marks:82,city:"Kanpur"},
  {roll:18,name:"Jai",age:18,marks:58,city:"Gonda"}
])
```

Figure: MongoDB execution screenshot — 1. CREATE DATABASE

2. CREATE COLLECTION

A collection named students is created to store the student documents.

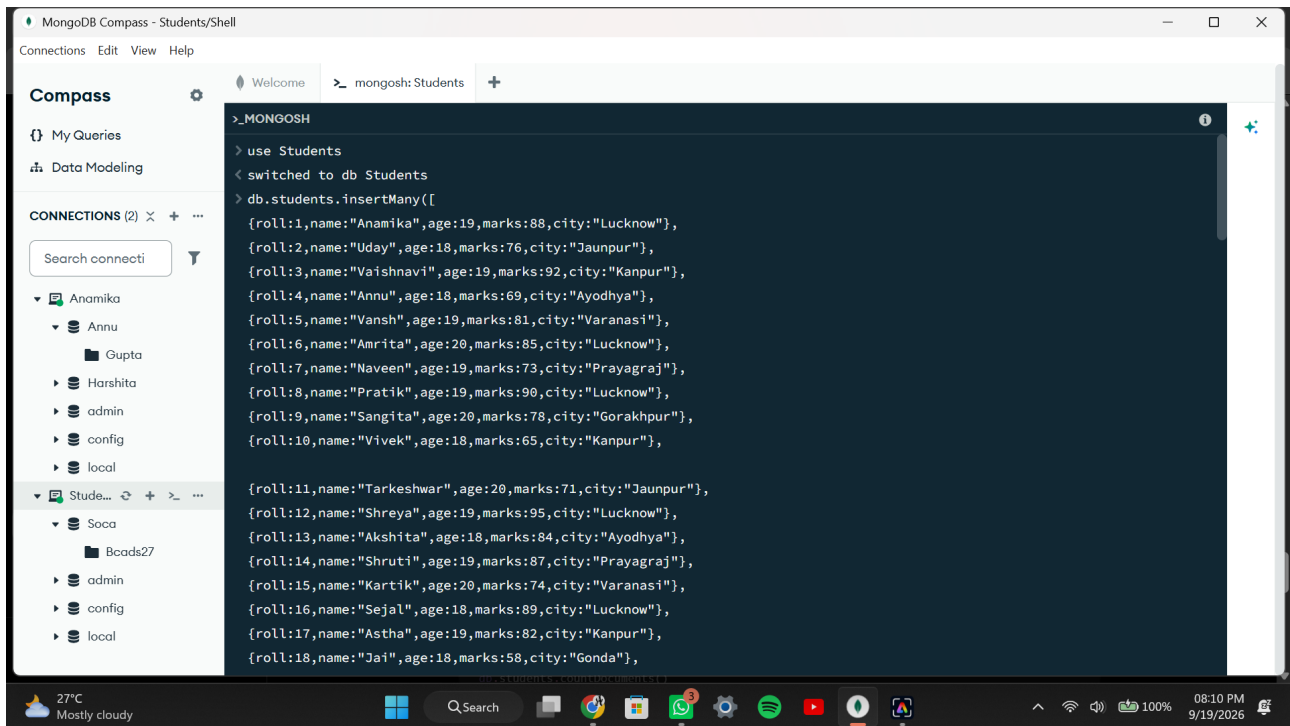


```
> use Students
switched to db Students
> db.students.insertMany([
  {roll:1,name:"Anamika",age:19,marks:88,city:"Lucknow"},
  {roll:2,name:"Uday",age:18,marks:76,city:"Jaunpur"},
  {roll:3,name:"Vaishnavi",age:19,marks:92,city:"Kanpur"},
  {roll:4,name:"Annu",age:18,marks:69,city:"Ayodhya"},
  {roll:5,name:"Vansh",age:19,marks:81,city:"Varanasi"},
  {roll:6,name:"Amrita",age:20,marks:85,city:"Lucknow"},
  {roll:7,name:"Naveen",age:19,marks:73,city:"Prayagraj"},
  {roll:8,name:"Pratik",age:19,marks:90,city:"Lucknow"},
  {roll:9,name:"Sangita",age:20,marks:78,city:"Gorakhpur"},
  {roll:10,name:"Vivek",age:18,marks:65,city:"Kanpur"},
  {roll:11,name:"Tarkeshwar",age:20,marks:71,city:"Jaunpur"},
  {roll:12,name:"Shreya",age:19,marks:95,city:"Lucknow"},
  {roll:13,name:"Akshita",age:18,marks:84,city:"Ayodhya"},
  {roll:14,name:"Shruti",age:19,marks:87,city:"Prayagraj"},
  {roll:15,name:"Kartik",age:20,marks:74,city:"Varanasi"},
  {roll:16,name:"Sejal",age:18,marks:89,city:"Lucknow"},
  {roll:17,name:"Astha",age:19,marks:82,city:"Kanpur"},
  {roll:18,name:"Jai",age:18,marks:58,city:"Gonda"}])
```

Figure: MongoDB execution screenshot — 2. CREATE COLLECTION

3. INSERT 50 STUDENTS

Multiple student documents are inserted into the students collection. The complete 50-record dataset is reproduced in the dataset section of this report.



The screenshot shows the MongoDB Compass interface with the 'Students' collection selected. The command window displays the following JavaScript code for inserting 50 records:

```
> use Students
< switched to db Students
> db.students.insertMany([
  {roll:1,name:"Anamika",age:19,marks:88,city:"Lucknow"},
  {roll:2,name:"Uday",age:18,marks:76,city:"Jaunpur"},
  {roll:3,name:"Vaishnavi",age:19,marks:92,city:"Kanpur"},
  {roll:4,name:"Annu",age:18,marks:69,city:"Ayodhya"},
  {roll:5,name:"Vansh",age:19,marks:81,city:"Varanasi"},
  {roll:6,name:"Amrita",age:20,marks:85,city:"Lucknow"},
  {roll:7,name:"Naveen",age:19,marks:73,city:"Prayagraj"},
  {roll:8,name:"Pratik",age:19,marks:90,city:"Lucknow"},
  {roll:9,name:"Sangita",age:20,marks:78,city:"Gorakhpur"},
  {roll:10,name:"Vivek",age:18,marks:65,city:"Kanpur"},
  {roll:11,name:"Tarkeshwar",age:20,marks:71,city:"Jaunpur"},
  {roll:12,name:"Shreya",age:19,marks:95,city:"Lucknow"},
  {roll:13,name:"Akshita",age:18,marks:84,city:"Ayodhya"},
  {roll:14,name:"Shruti",age:19,marks:87,city:"Prayagraj"},
  {roll:15,name:"Kartik",age:20,marks:74,city:"Varanasi"},
  {roll:16,name:"Sejal",age:18,marks:89,city:"Lucknow"},
  {roll:17,name:"Astha",age:19,marks:82,city:"Kanpur"},
  {roll:18,name:"Jai",age:18,marks:58,city:"Gonda"},
  {roll:19,name:"Rishabh",age:20,marks:70,city:"Jaunpur"},
  {roll:20,name:"Priya",age:19,marks:80,city:"Lucknow"},
  {roll:21,name:"Arjun",age:18,marks:75,city:"Ayodhya"},
  {roll:22,name:"Ishika",age:19,marks:88,city:"Prayagraj"},
  {roll:23,name:"Vishal",age:20,marks:72,city:"Varanasi"},
  {roll:24,name:"Neha",age:18,marks:91,city:"Lucknow"},
  {roll:25,name:"Ravi",age:19,marks:68,city:"Kanpur"},
  {roll:26,name:"Smita",age:20,marks:83,city:"Jaunpur"},
  {roll:27,name:"Adarsh",age:18,marks:77,city:"Lucknow"},
  {roll:28,name:"Divya",age:19,marks:86,city:"Ayodhya"},
  {roll:29,name:"Nikhil",age:20,marks:79,city:"Prayagraj"},
  {roll:30,name:"Sanya",age:18,marks:93,city:"Varanasi"},
  {roll:31,name:"Vikram",age:19,marks:67,city:"Lucknow"},
  {roll:32,name:"Anjali",age:20,marks:84,city:"Kanpur"},
  {roll:33,name:"Rohit",age:18,marks:76,city:"Jaunpur"},
  {roll:34,name:"Pooja",age:19,marks:89,city:"Lucknow"},
  {roll:35,name:"Amit",age:20,marks:73,city:"Ayodhya"},
  {roll:36,name:"Shweta",age:18,marks:92,city:"Prayagraj"},
  {roll:37,name:"Vishnu",age:19,marks:69,city:"Varanasi"},
  {roll:38,name:"Isha",age:20,marks:85,city:"Lucknow"},
  {roll:39,name:"Naman",age:18,marks:78,city:"Kanpur"},
  {roll:40,name:"Sneha",age:19,marks:90,city:"Jaunpur"},
  {roll:41,name:"Rishi",age:20,marks:71,city:"Lucknow"},
  {roll:42,name:"Priyanka",age:18,marks:87,city:"Ayodhya"},
  {roll:43,name:"Vikash",age:19,marks:82,city:"Prayagraj"},
  {roll:44,name:"Anshika",age:20,marks:75,city:"Varanasi"},
  {roll:45,name:"Rudra",age:18,marks:94,city:"Lucknow"},
  {roll:46,name:"Shreya",age:19,marks:66,city:"Kanpur"},
  {roll:47,name:"Arjun",age:20,marks:81,city:"Jaunpur"},
  {roll:48,name:"Divya",age:18,marks:74,city:"Lucknow"},
  {roll:49,name:"Nikhil",age:19,marks:88,city:"Ayodhya"},
  {roll:50,name:"Sanya",age:20,marks:70,city:"Prayagraj"}
])
```

Figure: MongoDB execution screenshot — 3. INSERT 50 STUDENTS

4. DISPLAY ALL STUDENTS

The display operation retrieves documents from the students collection and shows the stored student records.

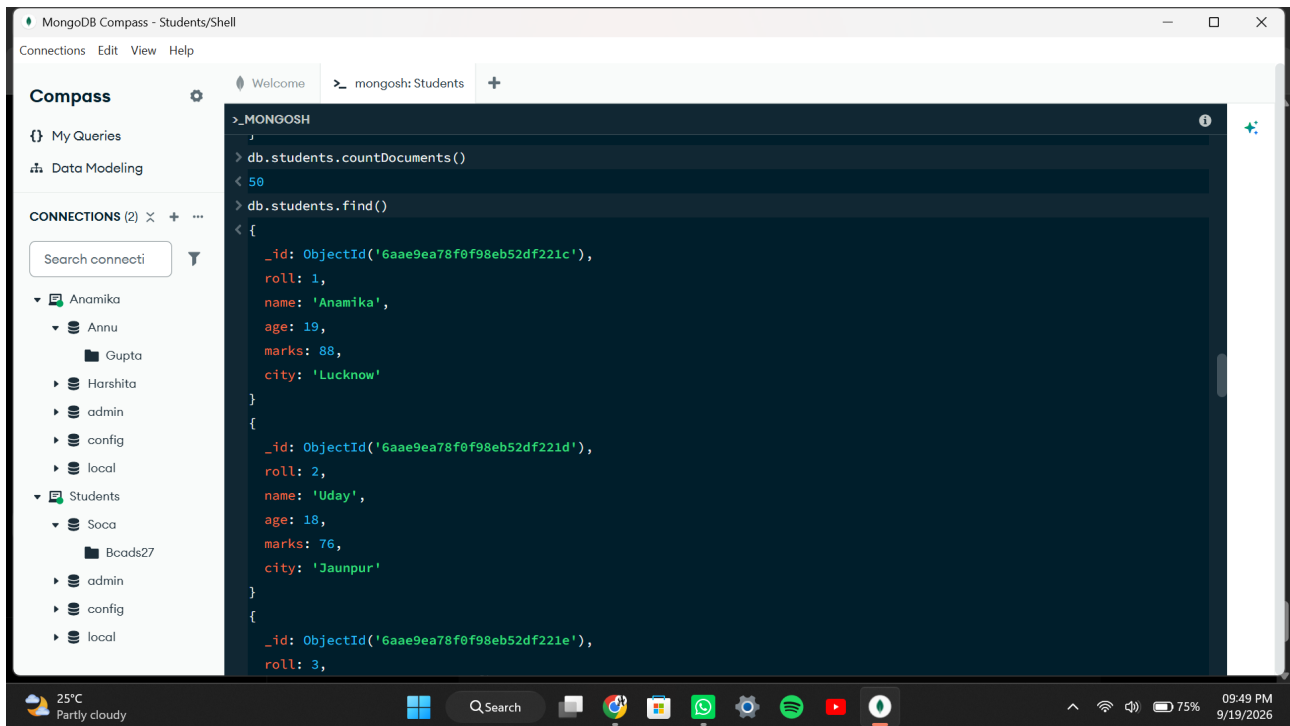


Figure: MongoDB execution screenshot — 4. DISPLAY ALL STUDENTS

5. EQUAL TO — \$eq

The \$eq comparison operator finds students whose marks are exactly equal to 88.

```
> db.students.find({marks: {$eq: 88}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2236'),
  roll: 27,
  name: 'Ananya',
  age: 19,
  marks: 88,
  city: 'Varanasi'
}
```

Figure: MongoDB execution screenshot — 5. EQUAL TO \$eq

6. GREATER THAN — \$gt

The \$gt comparison operator finds students whose marks are greater than 85.

```
> db.students.find({marks: {$gt: 85}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — 6. GREATER THAN \$gt

7. LESS THAN — \$lt

The \$lt comparison operator finds students whose marks are less than 60.

```
> db.students.find({marks: {$lt: 60}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Jai',
  age: 18,
  marks: 58,
  city: 'Gonda'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df223c'),
  roll: 33,
  name: 'Jai',
  age: 20,
  marks: 55,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224a'),
  roll: 47,
  name: 'Aman',
  age: 18,
  marks: 59,
  city: 'Gonda'
}
> db.students.find({marks: {$eq: 88}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — 7. LESS THAN \$lt

8. GREATER THAN OR EQUAL TO — \$gte

The \$gte comparison operator finds students whose marks are 90 or above.

```
> db.students.find({marks: {$gte: 90}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222e'),
  roll: 19,
  name: 'Harshita',
  age: 20,
  marks: 91,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — 8. GREATER THAN OR EQUAL TO \$gte

9. LESS THAN OR EQUAL TO — \$lte

The \$lte comparison operator finds students whose marks are 70 or below.

```
> db.students.find({marks: {$lte: 70}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221f'),
  roll: 4,
  name: 'Annu',
  age: 18,
  marks: 69,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2225'),
  roll: 10,
  name: 'Vivek',
  age: 18,
  marks: 65,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Jai',
  age: 18,
  marks: 58,
  city: 'Gonda'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222f'),
  roll: 20,
  name: 'Omkar',
  age: 19,
  marks: 63,
  city: 'Bareilly'
}
```

Figure: MongoDB execution screenshot — 9. LESS THAN OR EQUAL TO \$lte

10. UPDATE OPERATION

The update operation changes the marks of the student having roll number 5 to 95.

```
> _MONGOSH
}
> db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.students.find({roll:5})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
```

Figure: MongoDB execution screenshot — 10. UPDATE OPERATION

11. DELETE ONE OPERATION

The delete-one operation removes the document matching roll number 10.

```
}
> db.students.deleteOne({roll:10})
< {
  acknowledged: true,
  deletedCount: 1
}
> db.students.find({roll:10})
<
Students >
```

Figure: MongoDB execution screenshot — 11. DELETE ONE OPERATION

12. DELETE MANY OPERATION

The delete-many operation removes all students having marks below 60.

```
> db.students.deleteMany({marks:{$lt:60}})
< {
  acknowledged: true,
  deletedCount: 3
}
> db.students.find({marks:{$lt:60}})
<
Students > |
```

Figure: MongoDB execution screenshot — 12. DELETE MANY OPERATION

13. AND OPERATION — \$and

The \$and logical operator returns students satisfying both conditions: marks greater than 80 and age equal to 19.

```
>_MONGOSH
> db.students.find({
  $and:[
    {marks:{$gt:80}}, {age:19}
  ]
})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

Figure: MongoDB execution screenshot — 13. AND OPERATION \$and

14. OR OPERATION — \$or

The \$or logical operator returns students whose city is either Lucknow or Kanpur.

```
>_MONGOSH
> db.students.find({
  $or:[
    {city:"Lucknow"},{city:"Kanpur"}
  ]
})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

Figure: MongoDB execution screenshot — 14. OR OPERATION \$or

15A. SORT OPERATION — ASCENDING

The sort operation arranges students by marks from lowest to highest. Ascending order uses 1.

```
>_MONGOSH
> db.students.find().sort({marks:1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2237'),
  roll: 28,
  name: 'Bhaskar',
  age: 18,
  marks: 61,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2246'),
  roll: 43,
  name: 'Sarika',
  age: 19,
  marks: 62,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222f'),
  roll: 20,
  name: 'Omkar',
  age: 19,
  marks: 63,
  city: 'Bareilly'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224c'),
  roll: 49,
  name: 'Rahul',
  age: 19,
  marks: 64,
  city: 'Bareilly'
}
```

Figure: MongoDB execution screenshot — 15A. SORT OPERATION ASCENDING

15B. SORT OPERATION — DESCENDING

The sort operation arranges students by marks from highest to lowest. Descending order uses -1.

```
> _MONGOSH
> db.students.find().sort({marks:-1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df223e'),
  roll: 35,
  name: 'Krishna',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2235'),
  roll: 26,
  name: 'Anushka',
  age: 20,
  marks: 94,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — 15B. SORT OPERATION DESCENDING

16. LIMIT OPERATION

The limit operation displays only the first five documents returned by the query.

```
>_MONGOSH
> db.students.find().limit(5)
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
  name: 'Uday',
  age: 18,
  marks: 76,
  city: 'Jaunpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
```

Figure: MongoDB execution screenshot — 16. LIMIT OPERATION

17. SEARCH BY CITY

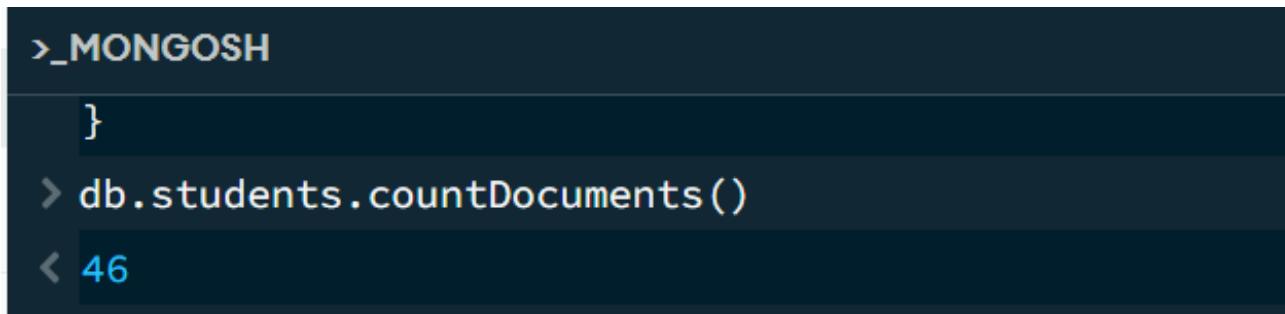
The city search finds all students whose city is Lucknow.

```
>_MONGOSH
> db.students.find({city:"Lucknow"})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2221'),
  roll: 6,
  name: 'Amrita',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
```

Figure: MongoDB execution screenshot — 17. SEARCH BY CITY

18. COUNT DOCUMENTS

The count operation counts the documents currently present in the collection after the delete operations.

A screenshot of a MongoDB shell session. The prompt is >_MONGOSH. The user has entered a closing curly brace } on the previous line. The current line shows the command > db.students.countDocuments(). The result of the command, 46, is displayed on the next line, preceded by a less-than sign <.

```
>_MONGOSH
}
> db.students.countDocuments()
< 46
```

Figure: MongoDB execution screenshot — 18. COUNT DOCUMENTS

MONGODB OPERATORS USED

The project demonstrates comparison and logical operators along with common result-processing methods used for MongoDB queries.

Operator / Method	Meaning	Example / Use
\$eq	Equal to	marks: {\$eq: 88}
\$gt	Greater than	marks: {\$gt: 85}
\$lt	Less than	marks: {\$lt: 60}
\$gte	Greater than or equal to	marks: {\$gte: 90}
\$lte	Less than or equal to	marks: {\$lte: 70}
\$and	All specified conditions must be true	\$and: [...]
\$or	At least one specified condition can be true	\$or: [...]
sort()	Sorts query results	Ascending / descending order
limit()	Limits the number of returned documents	First 5 documents
countDocuments()	Counts documents matching the collection query	Current document count

PROJECT WORKFLOW

The workflow followed in the project was:

1. Create/select the Students database.
2. Create the students collection.
3. Insert the 50 student documents.
4. Retrieve and display student records.
5. Apply comparison operators for filtering.
6. Update and delete selected records.
7. Apply AND and OR logical operators.
8. Sort and limit the returned results.
9. Search records by city.
10. Count the remaining documents after deletion.

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how ascending and descending order work, how comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte are used, and how logical operators such as \$and and \$or help in filtering data.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

PROJECT DETAILS

Student Name	Prince Kumar
Roll No.	16
Class	BCA DS 25
Department	BCA (DS & AI)
Teacher	Mr. Harendra Singh Rajpoot
Project	Student Management System Using MongoDB
Database	Students
Collection	students
Initial Records	50
Operations Demonstrated	Create, insert, display, filter, update, delete, sort, limit, search, count
Academic Session	2026–2027

End of Project Report